



ARQUITETURA DE SISTEMAS

AULA 6 - FUNDAMENTOS DE ESTILOS ARQUITETURAIS



INTRODUÇÃO

- ❑ O que são Estilos Arquiteturais: Padrões de organização de componentes de software, que possuem nome e características reconhecidas (semelhante a design patterns). Por exemplo, ao mencionar layered monolith para outro arquiteto experiente, ambos entendem a estrutura geral, pontos fortes e fracos, modelo de implantação típico e detalhes implícitos daquele estilo.
- ❑ Importância dos Estilos: Cada estilo arquitetural traz um “atalho mental” que comunica uma série de decisões e trade-offs já conhecidos. Arquitetos devem conhecer os estilos fundamentais, pois eles aparecem recorrentemente e formam base para padrões modernos mais complexos.



BIG BALL OF MUD (ANTI-PATTERN)

- ❑ Definição: Big Ball of Mud é um sistema sem arquitetura discernível, estruturado de forma caótica e “remendada” ao longo do tempo. O código se assemelha a um emaranhado (“spaghetti code”), onde componentes acessam indiscriminadamente uns aos outros, informações são globais ou duplicadas, e não há separação clara de responsabilidades.
- ❑ Consequências: Mudanças tornam-se arriscadas: qualquer alteração pode provocar efeitos colaterais imprevisíveis em outras partes do sistema. Essa falta de estrutura dificulta a evolução e manutenção, prejudicando testabilidade, escalabilidade e desempenho. Em implantações, pequenos “consertos rápidos” vão se acumulando e a aplicação vira uma grande colcha de retalhos, resistente a melhorias.
- ❑ Origem do Termo: O nome surgiu em um artigo de 1997 por Brian Foote e Joseph Yoder, descrevendo esse cenário comum em projetos sem governança de arquitetura. Embora nenhum time planeje deliberadamente criar um Big Ball of Mud, a pressão por entregas rápidas sem refatorações leva muitas aplicações triviais a crescerem desordenadamente até esse estado.
- ❑ Como Evitar: Adotar disciplina de modularização, definição clara de camadas e padrões de projeto desde o início. Revisões de código e refatorações contínuas ajudam a prevenir a “bola de lama”. Uma vez formada, extrair ordem do caos exige grandes esforços de refatoração e reestruturação.



ARQUITETURA CLIENTE/SERVIDOR (2 CAMADAS)

- ❑ Visão Geral: Modelo Client/Server de 2 camadas, onde a aplicação é dividida em cliente (geralmente responsável pela interface e parte da lógica) e servidor (responsável pelos dados ou funcionalidade central). O cliente consome serviços ou dados providos pelo servidor através de uma rede.
- ❑ Exemplos Clássicos:
 - ❑ Desktop + Database: Aplicações desktop (ex: em Delphi, VB) conectando-se diretamente a um servidor de banco de dados. A lógica de apresentação fica no cliente, enquanto a persistência e consultas complexas ficam no servidor de BD.
 - ❑ Browser + Web Server: Com a Web, o cliente passou a ser o navegador e o servidor web lida com lógica de negócio e persistência (geralmente falando com um banco de dados). O cliente é “magro” (thin-client, apenas interface HTML/JS) e a maior parte do trabalho ocorre no servidor web e no banco de dados.
- ❑ Características: Arquiteturas 2-tier são relativamente simples e adequadas para aplicações menores. A comunicação direta cliente-servidor facilita o desenvolvimento rápido. Entretanto, conforme a aplicação cresce, essa simplicidade pode virar limitação. Por exemplo, dificuldade de escalar ou modificar funcionalidades isoladamente, já que cliente e servidor estão fortemente acoplados.
- ❑ Considerações: Em Java (e outras linguagens), o modelo cliente/servidor tradicional evoluiu para incluir protocolos web (HTTP/REST) e drivers de banco (JDBC) no lado cliente. Ainda hoje, entender esse padrão é importante, pois muitos sistemas legados e mesmo arquiteturas modernas (como aplicações web simples) são essencialmente 2-tier (cliente web direto no servidor + BD).



ARQUITETURA TRÊS CAMADAS (THREE-TIER)

- ❑ Estrutura: A arquitetura Three-Tier introduz uma terceira camada intermediária entre cliente e servidor de dados. As três camadas típicas são: Apresentação (UI/Cliente), Aplicação (lógica de negócio, por exemplo um servidor de aplicações) e Dados (servidor de banco de dados). Cada camada é implantada separadamente e se comunica com as adjacentes, geralmente via rede.
- ❑ Motivações: Surge da necessidade de melhorar escalabilidade e manutenção em relação ao modelo 2-tier. Com uma camada dedicada à lógica de negócio, é possível separar preocupações de forma mais clara: UI trata da interação com usuário, a camada de aplicação processa regras de negócio e orquestra fluxos, e a camada de dados gerencia persistência. Essa separação modulariza o sistema, facilitando alterações e distribuição de carga.
- ❑ Exemplo (Web Enterprise): Um cliente web (navegador) faz requisições a um servidor de aplicação Java (por ex., rodando em um application server como JBoss/WebLogic), que processa a lógica e interage com um banco de dados relacional separado. O HTML é gerado no servidor de aplicação e retornado ao cliente. Essa era a arquitetura dominante no fim dos anos 1990 e 2000 para aplicações corporativas Java EE.



ARQUITETURA TRÊS CAMADAS (THREE-TIER)

- ❑ Vantagens
 - ❑ Maior escalabilidade: é possível escalonar separadamente o servidor de aplicação e o banco de dados conforme a demanda (e eventualmente a camada de apresentação em servidores web).
 - ❑ Melhoria na manutenibilidade: equipes podem trabalhar isoladamente em cada camada; mudanças na lógica de negócio não afetam diretamente a UI e vice-versa.
 - ❑ Além disso, permite reutilizar a lógica de negócio em diferentes interfaces (web, desktop, mobile) apenas alterando a camada de apresentação.
- ❑ Impacto na Plataforma: A influência dessa arquitetura foi tão grande que deixou marcas em tecnologias. Por exemplo, o Java incorporou suporte nativo à serialização de objetos pensando em trânsito de dados entre camadas distribuídas, já que se assumia que o futuro da computação seriam aplicações em três camadas distribuídas. Embora novas arquiteturas tenham surgido, o padrão 3-tier permanece relevante e inspira muitos designs atuais.



ARQUITETURA MONOLÍTICA VS. DISTRIBUÍDA

- ❑ **Arquitetura Monolítica:** Todo o código e funcionalidades do sistema estão unificados em um único deploy (uma única aplicação). Tipicamente, é uma aplicação contendo todas as camadas (UI, lógica, acesso a dados) dentro do mesmo processo ou pacote implantável (por ex., um arquivo WAR/EAR único). Monólitos são mais simples de desenvolver e implantar inicialmente: uma única base de código, transações locais fáceis e menor complexidade de infraestrutura.
- ❑ **Arquitetura Distribuída:** O sistema é quebrado em múltiplos componentes ou serviços que comunicam-se via rede. Exemplos incluem microserviços, SOA ou mesmo um sistema com vários processos distintos. Cada serviço é um deploy separado, podendo escalar independentemente e usar tecnologias diferentes. Isso traz flexibilidade e escalabilidade em cada componente, mas introduz desafios de comunicação remota, consistência de dados e orquestração.
- ❑ **Comparação - Trade-offs:** Monólitos favorecem consistência forte e simplicidade operacional: chamadas de função locais, transações ACID e logs centralizados. Em compensação, podem se tornar Big Ball of Mud se não modularizados, e escalar exige replicar todo o sistema. Já arquiteturas distribuídas oferecem maior escalabilidade e isolamento de falhas (um serviço pode cair sem derrubar todo o sistema), além de possibilitar deploys independentes por funcionalidade. No entanto, pagam o preço em complexidade: problemas de latência de rede, necessidade de lidar com falhas de comunicação, segurança entre serviços e maior esforço de deploy/monitoramento.



AS 8 FALÁCIAS DA COMPUTAÇÃO DISTRIBUÍDA

- ❑ 1) A rede é confiável
 - ❑ Falácia: assumimos que sempre haverá comunicação bem-sucedida.
 - ❑ Na realidade, conexões falham e pacotes se perdem. Nunca se deve depender 100% da rede; é preciso implementar retries, timeouts e lidar com indisponibilidade parcial dos serviços.
- ❑ 2) Latência zero
 - ❑ Falácia: achamos que chamadas remotas são instantâneas.
 - ❑ Toda comunicação via rede adiciona atraso significativo comparado a chamadas locais. Em microserviços, por exemplo, múltiplas requisições encadeadas podem introduzir centenas de milissegundos de latência acumulada. Deve-se projetar considerando atrasos de rede e otimizar o número de chamadas entre serviços.



AS 8 FALÁCIAS DA COMPUTAÇÃO DISTRIBUÍDA

- ❑ 3) Largura de banda infinita
 - ❑ Falácia: ignoramos o limite de throughput da rede.
 - ❑ A rede tem capacidade finita; transferir grandes volumes de dados ou muitas requisições simultaneamente pode saturar links e reduzir desempenho. Boas práticas incluem enviar apenas os dados necessários e usar mecanismos como pagination, filtros de campos, etc.
- ❑ 4) A rede é segura
 - ❑ Falácia: acreditamos que, por estar atrás de firewall ou VPN, o tráfego está protegido.
 - ❑ Em sistemas distribuídos, cada ponto de comunicação amplia a superfície de ataque. É necessário trabalhar a segurança de cada endpoint e validar chamadas, mesmo internas, pois ataques podem ocorrer dentro da rede confiável.



AS 8 FALÁCIAS DA COMPUTAÇÃO DISTRIBUÍDA

- ❑ 6) A topologia não muda
 - ❑ Falácia: supõe-se que a configuração da rede será estática.
 - ❑ Redes estão em constante mudança: roteadores podem ser reconfigurados, novos segmentos adicionados, etc. Alterações “invisíveis” na infraestrutura podem quebrar suposições de latência ou de rota e afetar a comunicação entre serviços. Arquitetos devem manter comunicação próxima com times de Ops e preparar sistemas para mudanças de rota, IPs ou região.
- ❑ 7) Existe um único administrador
 - ❑ Falácia: acreditar que apenas uma equipe controla todo o ambiente.
 - ❑ Em empresas grandes, há múltiplos administradores de rede, cada um gerindo partes distintas (sub-redes, servidores, cloud, etc.). Políticas diferentes podem entrar em conflito (por exemplo, restrições de firewall em um segmento). Deve-se coordenar com todas as partes envolvidas na infraestrutura ao projetar e depurar um sistema distribuído.



AS 8 FALÁCIAS DA COMPUTAÇÃO DISTRIBUÍDA

- ❑ 8) A rede é homogênea
 - ❑ Falácia: assumir que toda a rede usa a mesma tecnologia e se comporta de forma uniforme.
 - ❑ Em realidade, infraestruturas são heterogêneas: diferentes fornecedores de hardware de rede, diferentes protocolos e middleware. Componentes distintos podem não se integrar perfeitamente, resultando em comportamentos inesperados (pacotes perdidos, incompatibilidades). Deve-se projetar prevendo diferenças de ambiente e testar em cenários variados.



LOGGING DISTRIBUÍDO

- ❑ **Desafio:** Em uma aplicação monolítica, depurar um problema é relativamente simples: geralmente há um único log central onde todas as ações são registradas sequencialmente. Já em uma arquitetura distribuída, cada serviço gera seu próprio log, em formato potencialmente distinto e armazenado em local diferente. Seguir o rastro de uma transação ponta a ponta torna-se complexo: por exemplo, uma requisição de usuário pode passar por 5 serviços diferentes, cada um com seu log. Correlacionar esses eventos é trabalhoso.
- ❑ **Centralização de Logs:** Para mitigar, adotam-se ferramentas de log centralizado. Stack ELK (Elasticsearch, Logstash, Kibana), Splunk, Graylog, entre outros, coletam logs de múltiplas fontes e os agregam em um só lugar, permitindo buscas e análises unificadas. Com essas plataformas, é comum incluir um identificador único (ID de correlação) em cada requisição distribuída (propagado por todos os serviços) para conseguir concatenar os eventos relacionados nos logs.
- ❑ **Limites:** Mesmo com ferramentas, logging distribuído envolve grandes volumes de dados e complexidade. Deve-se planejar retenção e filtragem de logs (para não sobrecarregar o sistema de logging), garantir consistência nos formatos (padrões de mensagens de log em todos os serviços) e segurança (logs podem conter dados sensíveis). Além disso, a análise de causa raiz em sistemas distribuídos frequentemente exige também tracing distribuído (ex.: usando Zipkin, Jaeger) para complementar os logs e visualizar fluxos de chamadas entre serviços.



TRANSAÇÕES DISTRIBUÍDAS

- ❑ Transações em Monólitos: Em aplicativos monolíticos tradicionais, usufruímos de transações ACID locais: podemos iniciar uma transação, executar diversas operações (por ex., em diferentes tabelas ou componentes) e commitar ou dar rollback atomicamente. Os frameworks de persistência (JDBC, JPA) e servidores de aplicação dão suporte transparente a isso, garantindo consistência de dados com facilidade.
- ❑ Quebra da Atomicidade: Em arquiteturas distribuídas, quando uma operação de negócio envolve múltiplos serviços e bases de dados diferentes, não há como simplesmente usar COMMIT único para todos. Cada serviço gerencia sua própria persistência, então não existe um “coordenador global” simples para garantir atomicidade entre serviços. As transações distribuídas de dois-fases (2PC) existem teoricamente e até foram usadas (ex.: JTA em sistemas distribuídos), mas escalam mal e são complexas de implementar em larga escala.



TRANSAÇÕES DISTRIBUÍDAS

- ❑ Consistência Eventual: Como alternativa, sistemas distribuídos adotam o princípio de eventual consistency: os dados passam por etapas e ficam temporariamente inconsistentes entre serviços, mas espera-se que converjam para um estado consistente após algum tempo. Em vez de travar todo o sistema esperando todos confirmarem simultaneamente (como numa transação ACID global), cada serviço confirma sua parte e mecanismos de compensação tratam discrepâncias. Isso sacrifica consistência estrita em prol de disponibilidade e desempenho.
- ❑ Padrão Saga: Uma das abordagens mais comuns para gerenciar transações distribuídas é o padrão Saga. Uma Saga decompõe uma transação longa em uma sequência de passos locais em cada serviço, onde cada passo possui também uma ação de compensação (undo) caso algum passo posterior falhe. Por exemplo, se um fluxo envolve Serviço A, B e C, cada um executa sua operação e, se alguém falhar no meio, os anteriores executam operações de cancelamento para reverter o efeito parcial. Essa orquestração pode ser centralizada (um coordenador de saga chamando serviços) ou distribuído (via coreografia).



TRANSAÇÕES DISTRIBUÍDAS

- ❑ Transações BASE: Em contraste ao acrônimo ACID, em sistemas distribuídos fala-se em princípios BASE: Basic Available, Soft State, Eventual Consistency. Significa projetar serviços de forma que estejam basicamente sempre disponíveis, permitirem um estado temporariamente inconsistente (soft state), com a garantia de que eventualmente alcançarão a consistência. É um modelo mental útil para arquiteturas distribuídas: aceitar que algumas invariantes de dados serão resolvidas de forma assíncrona (ex.: crédito e débito em contas bancárias diferentes confirmados separadamente, reconciliados depois).



ATÉ A PRÓXIMA AULA